



Accurate Residues for Floating-Point Debugging

YUMENG HE[✉], University of Utah, USA

PAVEL PANCHEKHA, University of Utah, USA

Floating-point arithmetic is error-prone and unintuitive. Floating-point debuggers instrument programs to monitor floating-point arithmetic at run time and flag numerical issues. To do so, they estimate *residues*—the difference between actual floating-point and ideal real values—for every floating-point value in the program. A large literature has explored various approaches for computing these residues accurately (leading to few false reports, i.e., false positives and false negatives) and efficiently (leading to low overhead over uninstrumented execution). Unfortunately, the most efficient methods, based on “error-free transformations”, have a high rate of false positives, while the most accurate methods, based on high-precision arithmetic, are very slow. This paper builds on error-free-transformations-based approaches and aims to improve their accuracy while preserving efficiency.

To more accurately compute residues, this paper divides residue computation into two steps—rounding error computation and residue function evaluation—and shows how to perform each step accurately via careful improvements to the current state of the art. We evaluate on 44 large scientific computing workloads, focusing on the 14 benchmarks where prior tools produce false reports: our approach eliminates false reports on 10 benchmarks and substantially reduces them on the remaining benchmarks.

Moreover, we find that more complex numerical issues, such as those found in numerical analysis textbooks, require additional care, because floating-point debuggers suffer from *absorption*, in which two different machine-precision residues cannot both be computed accurately in a single execution. To address absorption, this paper introduces *residue override*, which re-executes the program multiple times, computing different residues in different executions and assembling a final “patchwork” execution where all residues are accurately computed. We evaluate on 169 standard benchmarks drawn from numerical analysis papers and textbooks, requiring only 3.6 re-executions on average. Among 34 benchmarks with false reports in the initial run, residue override is triggered on 29 of them and reduces false reports on 25 of them, averaging 7.1 re-executions.

CCS Concepts: • **Software and its engineering** → **Dynamic analysis**; **Software testing and debugging**; • **Mathematics of computing** → **Numerical analysis**.

Additional Key Words and Phrases: floating point, debugging, double-double, numeric, re-execute

ACM Reference Format:

Yumeng He and Pavel Panchekha. 2026. Accurate Residues for Floating-Point Debugging. *Proc. ACM Program. Lang.* 10, OOPSLA2, Article 394 (October 2026), 23 pages. <https://doi.org/10.1145/3839526>

1 Introduction

Scientific and mathematical software typically uses floating-point arithmetic to approximate arithmetic on real numbers. However, floating-point arithmetic is highly unintuitive [27]. Numerical problems like cancellation can invalidate whole computations, leading to potentially catastrophic consequences: financial losses [26, 29], election instability [34], and wartime casualties [33]. The programming languages community has therefore long worked on tools, such as floating-point debuggers, that make numerical programming safer and easier.

Authors' Contact Information: Yumeng He, University of Utah, Salt Lake City, USA, u1528477@utah.edu; Pavel Panchekha, University of Utah, Salt Lake City, USA, pavpan@cs.utah.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2475-1421/2026/10-ART394

<https://doi.org/10.1145/3839526>

Floating-point debuggers instrument a numerical program to detect, at run time, inaccurate operations that cause numerical issues. A variety of floating-point debugging approaches have been developed over the years, including high-precision shadow execution [3, 6, 7, 31], error-free transformations [1, 8, 14, 23, 36], and stochastic rounding [15]. At their core, these debuggers aim to estimate *residues*, meaning the difference between the value of a variable in the actual floating-point execution of a program and in an idealized real-number execution. By estimating these residues, floating-point debuggers can warn about values with large residues, detect unstable control flow, or track how erroneous operations affect program outputs.

The challenge is estimating residues accurately and efficiently. Floating-point debuggers using high-precision arithmetic, such as FpDebug [3], Herbgrind [31], and FPSanitizer [6, 7], are highly accurate but cause overheads of 100× or more over uninstrumented execution. Debuggers based on error-free transformations use only machine floating-point operations and are much faster [1, 8, 23, 36], but suffer a higher rate of false positives and negatives, with as many as 20% of warnings being false [23]. Stochastic debuggers require repeated re-executions to achieve accurate results [15]. To date, no floating-point debugging technique simultaneously achieves both high accuracy and low runtime overhead.

This paper demonstrates that the best of both worlds is achievable with a new approach to floating-point debugging, which we call RePo. RePo splits residue computation into two steps—*rounding error estimation* using error-free transformations and *residue function evaluation* using machine floating-point operations—and carefully addresses accuracy challenges for each step, such as accurate rounding errors for casts and accurate residue functions for multiplication operations. Compared to the current state of the art, which produces false reports on 14 of 44 scientific benchmarks, these improvements eliminate false reports on 10 benchmarks and substantially reduce them on the remaining benchmarks.

While accurate rounding errors and residue functions significantly improve debugging accuracy, addressing the most challenging numerical issues requires additional techniques. Specifically, machine-precision residues, which are critical for high performance, suffer from *absorption*, meaning that, for some programs, two different residues cannot both be accurately measured in the same execution. To address absorption, RePo introduces residue override, a technique that performs *multiple* executions of the program, computing a *different* set of residues accurately each time. A final execution then combines residues, measured in different executions, to produce an accurate floating-point debugging result. Specifically, RePo uses a carefully designed algorithm to resolve many instances of absorption in a small number of re-executions, even for complex numerical computations such as the internals of elementary functions. Across 34 of 169 standard benchmarks drawn from numerical analysis papers and textbooks, where RePo produces false reports in the initial run, residue override is triggered on 29 of them and reduces false reports on 25 of them, averaging 7.1 re-executions. In short, this paper’s contributions are:

- (1) Demonstrating that careful rounding error estimation and residue function evaluation significantly reduce false positive and false negative rates on large-scale scientific software (Section 3).
- (2) Demonstrating that, despite this, absorption makes it impossible to avoid false positives and false negatives on complex numerical benchmarks (Section 4).
- (3) Demonstrating the residue override technique, which computes residues accurately even in the presence of absorption through multiple re-executions (Section 5 and Section 6).

2 Worked Example

Consider the following computation over a double-precision variable x :

```
double a = x + 1;
double b = sqrt(x), c = sqrt(a);
double y = c - b;
double z = y * y;
```

For an extreme input like $x = 10^{99}$ this code outputs $z = 0$, whereas the true value of z is approximately $2.5 \cdot 10^{-100}$. A floating-point debugger should help the user diagnose this issue.

Residues. Floating-point debuggers work by computing the *residue* for each floating-point value in the program. Formally, the residue is the difference between two executions of the program: the actual value computed in a floating-point execution of the program, and the ideal value computed in a hypothetical real-number execution. Every floating-point operation in the program has an associated residue; Table 1 shows the residue value for each operation in our running example. For example, for the addition $a = x + 1$, the actual floating-point result is 10^{99} while the ideal real-number result is $10^{99} + 1$, so the residue is 1. Floating-point debuggers use these residues to warn users about potential numerical issues. For example, since the residue for z , roughly $2.5 \cdot 10^{-100}$, is large relative to its computed value of 0, a floating-point debugger can issue a warning that the value of z is unreliable and subject to numerical error. The exact threshold used to trigger warnings varies across debuggers [6, 31], but accurate residue computation is essential for all of them.

Various techniques for estimating residues exist. Early floating-point debuggers like Herbrind [31] attempted to directly compute the ideal value using MPFR [16], and then subtract it from the actual value. This required hundreds or even thousands of bits of precision to obtain accurate results. Other debuggers, such as EFTSanitizer [8], attempt to compute residues directly using *error-free transformations*, which are clever sequences of machine floating-point operations that capture the error of other machine floating-point operations. This requires less precision and is thus much faster, but the resulting residues are less accurate, causing many false positives and false negatives compared to the MPFR-based approach. There are other approaches as well; for example, Verrou [15] estimates residues by aggregating many randomized executions of the program. In contrast, this paper separates residue computation into two distinct tasks: accurate estimation of *rounding errors*, at which error-free transformations excel, and their aggregation via *residue functions*.

Rounding Errors and Residue Functions. The residue e_z of a floating-point operation $\hat{z} = \hat{f}(\hat{x}, \hat{y})$ comes from two effects. First, the real value $f(\hat{x}, \hat{y})$ might not be exactly representable in floating-point, so $f(\hat{x}, \hat{y})$ must be rounded. That rounding introduces the *rounding error* $\mu_z = f(\hat{x}, \hat{y}) - \hat{f}(\hat{x}, \hat{y})$. Second, since $\hat{x}$ and $\hat{y}$ are themselves floating-point values, they are affected by their own rounding errors, and recursively by even earlier rounding errors of earlier intermediate values. Conceptually,

Table 1. A step-by-step demonstration of how an ideal floating-point debugger should work for the running example. Each operation has an actual value, an ideal value, and a residue, which is the difference between the two. In this case, the residues for b and c are both large and of similar magnitude. The debugger warns the user about values (here, y and z) whose residues are relatively large compared to their actual values.

Operation	Actual value	Ideal value	Residue
$a=x+1$	1.00000000e+99	1.00000000...00100...e+99	1
$b=\text{sqrt}(x)$	3.16227766e+49	3.16227766...29518...e+49	1.31447527...18810...e+32
$c=\text{sqrt}(a)$	3.16227766e+49	3.16227766...29676...e+49	1.31447527...18968...e+32
$y=c-b$	0	1.58113883...14719...e-50	1.58113883...14719...e-50
$z=y*y$	0	2.49999999...99875...e-100	2.49999999...99875...e-100

Table 2. A step-by-step demonstration of how RePo uses residue override to resolve absorption. The first run produces two false negatives at the operations computing y and z because absorption occurred when computing e_c . The residue override mechanism therefore detects the operations whose rounding errors have the largest impact on the cancelled residues—in this case, the two square-root operations—and silences them in the next execution (Run 2). As a result, the residue for $\sqrt{x}$ becomes 0, while the residue for $\sqrt{x} + 1$ becomes the previously absorbed small term (about 1.6×10^{-50}). At this run, although we correctly measure the residues for the last two operations (which were previously false negatives), the residues at the two square-root operations become much worse. Therefore, in the final execution (Run 3), residue override stops silencing the two square-root operations and overrides the residues of the last two operations with the previously correctly computed values. Now every operation has an accurate residue.

Operation	Residue (Run 1)	Residue (Run 2)	Residue (Run 3)
$a=x+1$	1.0000000000000000e+00	1.0000000000000000e+00	1.0000000000000000e+00
$b=\text{sqrt}(x)$	1.3144752779492117e+32	0	1.3144752779492117e+32
$c=\text{sqrt}(a)$	1.3144752779492117e+32	1.5811388300841897e-50	1.3144752779492117e+32
$y=c-b$	0	1.5811388300841897e-50	1.5811388300841897e-50
$z=y*y$	0	2.5000000000000000e-100	2.5000000000000000e-100

the residue e_z aggregates rounding errors as $e_z[\mu_1, \dots, \mu_x, \mu_y, \mu_z]$. It is computed via a residue function $e_f(\hat{x}, \hat{y}, \hat{z}, \mu_z, e_x, e_y)$ together with the original floating-point operation. The process of computing rounding errors is relatively straightforward, using *error-free transformations* that estimate rounding errors μ_z for elementary operations such as addition/subtraction, multiplication, division, and square root. Residue functions, however, are more challenging.

Previous floating-point debuggers that use error-free transformations, such as EFTSanitizer and Shaman [8, 14], focus on simply detecting whether a program is accurate or not. This is a coarser objective than accurately computing all residues in the program, and both tools thus use correspondingly coarse, *first-order* residue functions. For example, consider the squaring operation, $\hat{z} = \hat{y} \hat{\cdot} \hat{y}$, in our example program. The residue e_z is computed as

$$z - \hat{z} = y \cdot y - \hat{y} \hat{\cdot} \hat{y} = (\hat{y} + e_y) \cdot (\hat{y} + e_y) - (\hat{y} \cdot \hat{y} - \mu_z) = \mu_z + 2\hat{y}e_y + e_y^2,$$

but a first-order residue function drops the e_y^2 term. While these first-order residue functions typically compute *some* residues accurately, they can lead to significant inaccuracies as errors interact and compound. For example, in the running example, the actual value of y is zero even though its residue e_y is nonzero. With the first-order residue function, e_z is therefore computed to be zero, whereas an accurate residue function yields a nonzero value. More generally, we find that computing residue functions accurately significantly reduces the number of false positive and false negative warnings on a suite of standard scientific computing benchmarks.

Absorption. Even with maximally accurate rounding errors and residue functions, some inaccuracies remain in the most challenging programs, because of a phenomenon we name *absorption*. Consider two different residues in our running example: e_c and e_y , which are the residues of operations computing $c = \sqrt{a}$ where $a = x + 1$, and $y = c - b$ where $b = \sqrt{x}$. The residue e_c is affected by the rounding error μ_c of the square root operation and the rounding error μ_a of the addition operation. Moreover, the effect of μ_c is much, much larger than the effect of μ_a —by about 50 orders of magnitude! Accurately measuring the residue e_c thus requires accurately computing μ_c and accurately incorporating it with μ_a into e_c .

However, since μ_c is so much larger than the propagated contribution of μ_a , this forces e_c to round away the contribution of μ_a entirely. We say that μ_c *absorbs* this contribution; it is an unavoidable

consequence of storing the residue e_c in finite machine precision while attempting to compute it accurately. The problem comes later, when attempting to compute $e_y = e_c - e_b$.¹ The ideal result is roughly $\mu_a/2c$, but because μ_c entirely absorbed this contribution when computing e_c , the actual computed residue e_y becomes 0. In other words, attempting to compute e_c accurately results in e_y being computed inaccurately. Alternatively, it is possible to compute e_y accurately by simply ignoring the contribution of μ_c to e_c and μ_b to e_b . Then e_c will be computed to be $\mu_a/2c$, while e_b will be computed to be 0, since we effectively ignore all contributions to it. This results in highly inaccurate residues for e_c and e_b , but it *does* result in an accurate value for e_y : $\mu_a/2c - 0 = \mu_a/2c$. In short, absorption means that either e_c or e_y can be computed accurately, but not both in the same execution.

Residue Override. The impossibility of accurately measuring both e_c and e_y in the same execution in error-free-transformations-based debuggers, due to the problem of absorption, suggests an alternative approach: measure each accurately in separate executions and then combine the results. RePo’s *residue override* technique does exactly this. During the first execution, it *detects* residues that cannot be accurately computed in the same execution. During a second execution, it *silences* the rounding errors with the greatest influence on one set of residues by setting those rounding errors to 0, and *probes*, meaning records, the now accurately computable residue. Finally, in a third execution, the rounding errors are no longer silenced, but the inaccurate residues are *overridden* with the more accurate probed values. This final execution has accurate residues for each value and thus produces fewer false positives and false negatives. Table 2 shows how residue override in RePo resolves the problem of absorption and produces correct residues for all operations in the running example. Our results, described in Section 7, show that RePo works remarkably well even on challenging programs, greatly reducing the number of false positives and false negatives on a standard suite of benchmarks drawn from textbooks and numerical analysis papers.

3 Accurate Machine-Float Residues

Floating-point debuggers rely on accurate residues to determine when to warn users about numerical issues. To accurately compute residues for all floating-point intermediate values, RePo aims to estimate rounding errors for each floating-point operation and then use those rounding errors to evaluate the residue function for each operation. RePo’s implementation of these two steps is based on EFTSanitizer, and the overall approach is similar. However, our initial evaluation showed that EFTSanitizer commonly produced false positives and false negatives. By refining rounding error estimation and residue function evaluation, RePo produces dramatically more accurate residues, with few false positives and false negatives across a range of scientific software.

3.1 Accurate Rounding Error Estimation

Rounding error estimation means computing, for an operation $f(\hat{x}, \hat{y})$ on floating-point inputs $\hat{x}$ and $\hat{y}$, the difference between the exact real result and the actual floating-point result. In other words, it estimates the error introduced *by that operation*. We refer to this as rounding error *estimation* because for some operations, such as division and square root, the rounding error is not exactly representable in floating-point, and is therefore itself rounded. For the basic arithmetic operations—addition, subtraction, multiplication, division, and square root—RePo estimates rounding error identically to EFTSanitizer, using error-free transformations.

However, RePo also estimates rounding error for casts between data types. When a 64-bit floating-point value is cast to a 32-bit floating-point value, for example, the resulting 32-bit value may not be exactly the same as the original 64-bit value, since fewer bits are available, and thus rounding

¹The original formula should be $e_y = \mu_y + e_c - e_b$ but we elide μ_y because it is equal to 0.

error occurs. RePo instruments all such cast operations and measures error by casting the 32-bit result back to 64 bits and subtracting. The subtraction is exact by Sterbenz's law. Casts from 32 to 64 bits are exact and do not need to be instrumented.

Additionally, RePo detects and specially handles a numerical trick for rounding double-precision values. In standard round-to-nearest-even rounding, when $|x| < 2^{52}$, adding and then subtracting the value $c = 1.5 \cdot 2^{52}$, as in $(x + c) - c$, rounds the value x to its nearest integer. This works because adding c rounds off all fractional bits of x . Since c is itself an integer, subtracting it restores the value of x with the fractional bits rounded away. Importantly, since the purpose of this trick is to round x to an integer, the rounding errors of the individual addition and subtraction operations are irrelevant: they are purposeful, not accidental, and measure the fractional bits of x , which are intentionally discarded. RePo detects this specific pattern (as well as the equivalent single-precision pattern, and variants in which the subtraction becomes the addition of $-c$) and sets the rounding error for both operations to zero. This particular improvement in rounding error handling is responsible for a dramatic reduction in false positives and false negatives inside standard library implementations of trigonometric functions like $\sin$ and $\cos$.

3.2 Accurate Residue Functions

The residue function for an operation $f(\hat{x}, \hat{y})$ combines the rounding error μ_z with the input residues e_x and e_y to compute the residue e_z of z . We observed that the residue functions used by EFTSanitizer contained several simplifications and inaccuracies. Correcting these issues dramatically reduces false positives and false negatives on large-scale scientific software.

First, the EFTSanitizer implementation included two typos in the residue functions. The first bug affects the residue computation for subtraction, which should be $\mu_z + e_x - e_y$, but instead computes $\mu_z + e_x + e_y$. The second, similar bug affects division, where the correct formula is $(e_x - \mu_z - \hat{z} \cdot e_y) / (\hat{y} + e_y)$, but where EFTSanitizer adds instead of subtracting the μ_z term. We reported both bugs to the EFTSanitizer authors, who confirmed the issues; the EFTSanitizer paper lists the correct (not buggy) formulas. Fixing these bugs, especially the subtraction bug, leads to dramatic reductions in false positives and false negatives. Since the paper in fact presents the correct formulas, all comparisons against EFTSanitizer in this paper use a version with both typos corrected.

Second, EFTSanitizer discards the "higher-order error" term in its residue function for multiplication. That is, the residue e_z in $\hat{z} = \hat{x} \cdot \hat{y}$ should be $\mu_z + \hat{y}e_x + \hat{x}e_y + e_xe_y$, but EFTSanitizer discards the e_xe_y term. Restoring it is especially important when $\hat{x}$ or $\hat{y}$ suffers from cancellation; in these cases $\hat{x} = 0$ but e_x is nonzero. If this affects both $\hat{x}$ and $\hat{y}$, the e_xe_y term becomes the only nonzero term in the residue function, and dropping it would cause the debugger to lose track of the cancellations entirely. Restoring the e_xe_y term therefore also leads to a dramatic reduction in false positives and false negatives.

Third, for absolute value, EFTSanitizer's residue function suffers from floating-point error. The absolute value operation is exact, so it introduces no rounding error, but it must still have a residue function to propagate the effects of input error. EFTSanitizer computes the residue as $|\hat{x} + e_x| - |\hat{x}|$. However, this is inaccurate when e_x is small relative to $\hat{x}$: the $\hat{x} + e_x$ computation may round away the small residue e_x , causing the final residue to be incorrectly computed as 0. Instead, RePo compares the signs of $\hat{x}$ and $\hat{x} + e_x$. If both have the same sign, RePo computes the residue as $\text{copysign}(1, \hat{x}) \cdot e_x$. When their signs differ, e_x and $\hat{x}$ must have similar magnitude, and the EFTSanitizer formula is used. The effect of this change is smaller than that of the previous ones, but it still affects several benchmarks.

Finally, for square-root operations, EFTSanitizer uses the residue function $(e_x + \mu_z)/(2 \cdot \hat{z})$. However, the $2 \cdot \hat{z}$ term is actually an approximation of $\sqrt{\hat{x}} + \sqrt{\hat{x} + e_x}$, which RePo uses instead; the two terms differ when e_x is large. We did not observe any changes in false positives or false negatives when switching to the more accurate formula in RePo, but experience with the previous residue function corrections suggests that using the accurate residue function is worthwhile.

3.3 Overflow and Underflow

The error-free transformations used by RePo and previous machine-float residue debuggers do not apply when inputs are outside the normal numerical range. RePo therefore does not attempt to handle overflow and underflow, which in any case correspond to range errors rather than rounding errors. To prevent overflow or underflow from causing false positives or false negatives for rounding errors, we guard rounding error estimation and residue function computation so that out-of-range inputs produce out-of-range outputs, and no warnings are generated for such values.

In our evaluation, we compare RePo against an MPFR-based debugger. MPFR has an expanded exponent range and can therefore represent values that overflow in machine precision. To ensure a fair comparison, we modify the MPFR-based debugger to treat out-of-range values consistently with RePo and to suppress warnings on overflow.

3.4 Results on Scientific Benchmark Suites

To demonstrate the accuracy impact of RePo's rounding error estimation and residue function implementations, we evaluate it on three standard scientific software suites: the NAS Parallel Benchmarks (8 benchmarks) [2], the Rodinia suite (18 benchmarks) [4], and the Polybench suite (30 benchmarks) [28], discarding programs that do not use floating-point arithmetic (5 in Rodinia and 2 in Polybench). We use a simple threshold where any residue greater than or equal to 2^{45} ULPs counts as a numerical warning² and record the exact operations at which warnings are raised. To count false positives and false negatives, we generalized RePo to offer pluggable residue backends and implemented an MPFR backend with a benchmark-specific working precision between 128 and 4096 bits, to act as a ground truth. We discard programs that run out of memory with MPFR even at 128 bits of precision (5 in total), leaving 44 benchmarks.

Overall, RePo has false positives or negatives—false reports—on just 4 of the 44 benchmarks. To compare to the current state of the art, we re-implemented EFTSanitizer, with the subtraction and division bugs noted above fixed,³ as yet another pluggable backend. The EFTSanitizer reimplementation reports false positives or false negatives on 14 benchmarks, far more and a strict superset of those reported by RePo; Figure 1 plots the results in detail. Even when both debuggers report false positives or false negatives on a benchmark, RePo reports far fewer. On `particlefilter`, RePo has just one false report, compared to over a hundred thousand for EFTSanitizer, and on `mg` and `myocyte` it likewise produces orders of magnitude fewer. Across all benchmarks, RePo produces about 38× fewer false reports than EFTSanitizer, and on no individual benchmark does it produce more. In fact, only on the `gramschmidt` benchmark do RePo and EFTSanitizer have even remotely similar false report counts: 8 448 for EFTSanitizer and 5 686 for RePo. We manually examined these false reports and found that many have a ground truth residue values close to the 2^{45} ULP threshold, suggesting that most of these remaining false reports come more from threshold effects rather than any remaining inaccuracy in the computed residues. In short, RePo achieves near-perfect results on

²We chose this value to match EFTSanitizer.

³We also tested the original EFTSanitizer subtraction and division residue functions but, as expected, they caused dramatically more false reports.

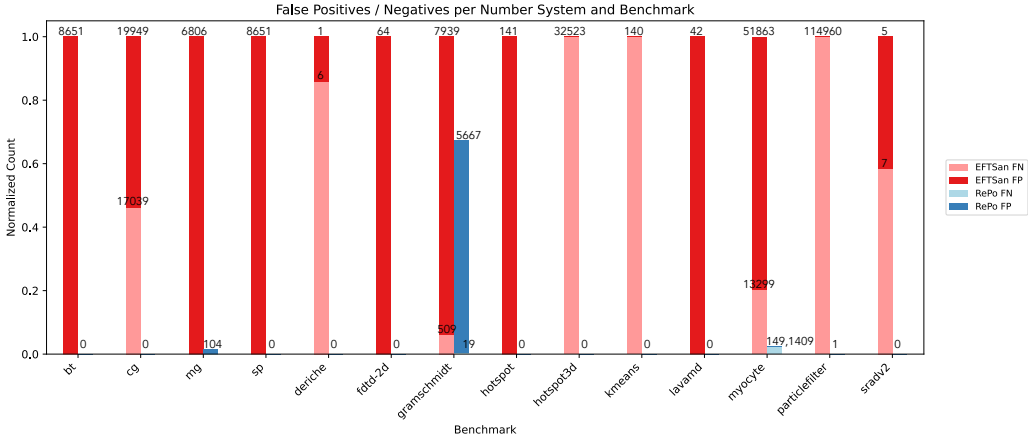


Fig. 1. Number of false reports (false positives and false negatives) for the baseline residue algorithm (an EFTSanitizer reimplementation with the subtraction and division bugs fixed) and our approach (RePo). Only benchmarks with false positives or false negatives are shown; all bars are normalized so that the total bar height of EFTSanitizer is 1.0. RePo produces dramatically fewer false reports than EFTSanitizer; only four benchmarks exhibit any false reports, and on three of them RePo has orders of magnitude fewer. On the remaining benchmark, *gramschmidt*, the false reports arise from residues clustering near the 2^{45} ULP threshold rather than from significant inaccuracies in the computed residues.

standard scientific benchmark suites by carefully designing rounding error estimation and residue function implementations for maximum accuracy.

4 Absorption and Residue Override

These results might suggest that accurately computing rounding errors and residue functions is sufficient for nearly perfect floating-point debugging. While this may hold for already relatively accurate standard scientific benchmarks, it is far less true for challenging numerical cases due to an issue we name *absorption*. In fact, without proper treatment, not only can the RePo debugger be inaccurate on such inputs (see Section 7), but *no* floating-point debugger can compute these residues accurately in a single execution without using prohibitively high precision.

4.1 Absorption

Absorption is the phenomenon that a numerical program may contain pairs of residues for which no single execution can measure both residues accurately. This occurs when a residue e_i is influenced by two different rounding errors μ_{i^*} and μ_ℓ , and these rounding errors contribute to e_i with dramatically different magnitudes; for example, μ_{i^*} 's contribution to e_i may be many orders of magnitude larger than μ_ℓ 's. Since e_i has limited precision, it cannot accurately represent both contributions. A typical floating-point debugger therefore represents only the larger one, μ_{i^*} , while rounding away the smaller one, μ_ℓ . As a result, e_i itself may be computed accurately.

However, some later residue e_k may combine e_i with another residue e_j in such a way that the contribution of μ_{i^*} cancels, either with its own or with another rounding error μ_{j^*} 's contribution to e_j . In this case, μ_{i^*} and μ_{j^*} no longer contribute to e_k . The smaller rounding error μ_ℓ may still contribute to e_k , but because it was not represented in e_i , that contribution is lost, leaving e_k computed inaccurately.

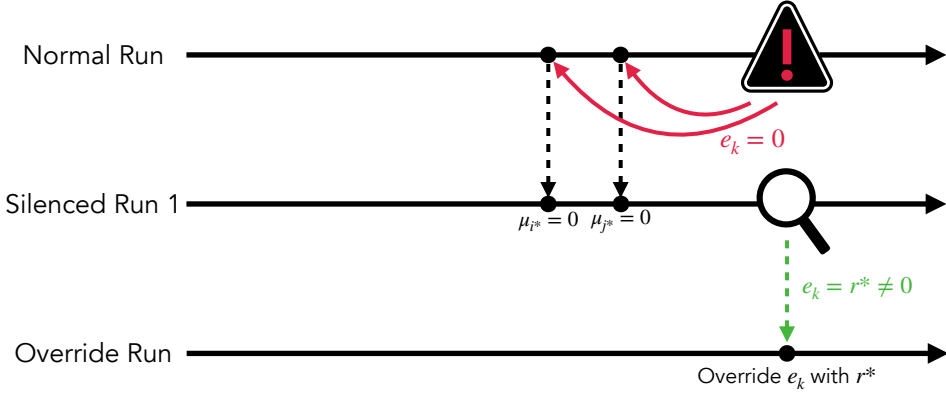


Fig. 2. The residue override framework estimates more precise residue values by executing the target program three times in total. The first execution *detects* cancellation ($e_k = 0$) and identifies the leading terms that cancel (i^* and j^*). The second execution *silences* operations i^* and j^* , ignoring their rounding errors when computing residues ($\mu_{i^*} = \mu_{j^*} = 0$). This results in a more accurate, non-zero value $r^* \neq 0$ for the residue e_k . The third execution then re-enables i^* and j^* , but overrides the residue e_k with r^* , replacing the cancellation with a more accurate residue value.

This observation leads to an *impossibility* result: *no* floating-point debugger that stores residues using machine floating-point values can accurately compute all residues in all situations, especially in challenging numerical code involving multiple cancellations or the internals of library functions.

At its core, absorption is simply a consequence of limited precision in representing residues, and may initially appear unsurprising. What *is* surprising, however, is that it is often possible to measure e_k accurately, though at the cost of measuring e_i inaccurately. Imagine that, instead of storing μ_{i^*} 's large contribution, e_i stored only the smaller contribution μ_ℓ . This would make e_i highly inaccurate, but it would preserve μ_ℓ 's contribution for the later computation of e_k . In many challenging numerical programs, this would allow e_k to be computed accurately.

In short, *both* e_i and e_k can be computed accurately in this scenario, but not within the same execution. This observation suggests a way around the impossibility result: perform multiple executions, each measuring different subsets of residues. We call this solution residue override.

4.2 Residue Override

The basic idea of residue override is illustrated in Figure 2, which uses the same assumption introduced earlier: residue e_k suffers from cancellation when it combines residues e_i and e_j such that their largest contributors cancel. The method consists of three executions of the target program. The first two executions measure different sets of residues, and the last execution combines them.

In the figure, the first execution, labeled “Normal Run”, measures residues e_i and e_j accurately, but as a result cannot measure e_k accurately. The second execution, labeled “Silenced Run 1” in the figure, *silences* rounding errors μ_{i^*} and μ_{j^*} , excluding their contributions when computing e_i and e_j . This means that the silenced run measures e_i and e_j inaccurately, but may compute an accurate value for e_k , which the figure labels r^* . We say that this second run also *probes* e_k , meaning that it records the value that e_k takes in this run. Note that silencing μ_{i^*} and μ_{j^*} affects only the debugger-computed residues, not the actual program values, and therefore cannot change control flow or other program behavior. The third execution, labeled “Override Run” in the figure, no longer silences μ_{i^*} and μ_{j^*} , and thus again measures e_i and e_j correctly. However, instead of computing e_k

normally—and thus inaccurately!—this final run *overrides* its value with r^* , the value computed during the silenced run. In other words, the final override run obtains accurate values for e_i , e_j , and e_k .

Making this basic idea work requires a critical step: *detecting* when absorption occurs, in which case e_k must be estimated separately from e_i and e_j and can be measured accurately only by silencing the dominant rounding errors μ_{i^*} and μ_{j^*} . RePo achieves this through a simple mechanism. Every floating-point operation executed during the program is assigned an “operation ID”, which is just an auto-incrementing 64-bit number. The program is assumed to be deterministic, so these operation IDs serve as stable identifiers across multiple executions.

The debugger stores, for every floating-point value i in the program, not only its residue e_i but also the operation ID i^* of the rounding error μ_{i^*} that makes the largest contribution to e_i . When an operation $\hat{z} = \hat{f}(\hat{x}, \hat{y})$ computes a residue e_z , RePo monitors the numerical error of the residue computation. If large numerical error is detected, absorption is assumed to prevent the output residue e_z and the input residues e_x and e_y from simultaneously being computed accurately. The largest contributors μ_{i^*} and μ_{j^*} to e_x and e_y are then silenced on the next run. Silencing the largest contributors to e_x and e_y makes those residues less accurate, but also allows smaller contributors to e_x and e_y to be represented, which may enable e_z to be computed more accurately.⁴

Formally, every residue function in RePo for an operation $\hat{z} = \hat{f}(\hat{x}, \hat{y})$ is structured as $e_z = A\mu_z + Be_x + Ce_y$, where A , B , and C can depend on $\hat{x}$, $\hat{y}$, and their residues e_x and e_y . Since A , B , and C can depend on e_x and e_y , this is not a purely “linear” or “first-order” residue function. For example, for a multiplication operation $\hat{z} = \hat{x} \cdot \hat{y}$, we define

$$e_z = 1 \cdot \mu_z + \left(\hat{y} + \frac{1}{2}e_y \right) e_x + \left(\hat{x} + \frac{1}{2}e_x \right) e_y;$$

this expands to the expected $\mu_z + \hat{y}e_x + \hat{x}e_y + e_xe_y$, which includes a higher-order error term.

Separating the $A\mu_z$, Be_x , and Ce_y terms makes it easy to define the “largest contributor” to e_z . We compare the magnitudes of these three terms. If $|A\mu_z|$ is the largest, then μ_z is the largest contributor to e_z . If $|Be_x|$ or $|Ce_y|$ is largest, then the largest contributor to e_z is the largest contributor to e_x or e_y , respectively. In the case of ties, the order is e_z , e_x , and then e_y . Through this mechanism, every residue has an assigned largest contributor.⁵

If high numerical error is detected while computing e_z , the largest contributors to e_x and e_y are then silenced and e_z is probed on the next run. A final override run then combines these measurements to provide accurate residues for floating-point debugging. This *detect-silence-probe-override* sequence is effective at reducing false positive and false negative warnings on a number of challenging numerical benchmarks, as discussed in Section 7. We now turn to the question of how RePo implements this basic algorithm.

5 Implementing Residue Override

Section 4 illustrated the basic idea of residue override; we now describe how to implement it in practice.

5.1 Tracking the Largest Error Contributor

Section 4 describes how to identify the largest contributor to a residue by expressing

$$e_z = A\mu_z + Be_x + Ce_y$$

⁴Of course, more complicated situations can arise; these are discussed in Section 5.

⁵For exactly computed values, which have no contributing rounding errors, a dummy largest contributor (-1) is assigned.

and comparing the magnitudes of the three terms. To implement this mechanism, RePo stores the identity of the largest contributor in a field `maxErrOp` within each residue.

Let `absIntroErr` denote $|A\mu_z|$, `absAmpErr1` denote $|Be_x|$, and `absAmpErr2` denote $|Ce_y|$. The `maxErrOp` field is assigned as follows:

```
if (absIntroErr >= max(absAmpErr1, absAmpErr2))
    e_z->maxErrOp = curOp;
else if (absAmpErr1 >= absAmpErr2)
    e_z->maxErrOp = e_x->maxErrOp;
else
    e_z->maxErrOp = e_y->maxErrOp;
```

Thus, if the local rounding error dominates, the current operation is recorded; otherwise, the dominant contributor is inherited from the input with the larger propagated error. Each residue therefore carries the identity of the operation that contributes most to its error.

5.2 Detection of Absorption

In the residue override framework, the detection phase determines when residue override should be triggered. The silence, probe, and override phases are largely self-contained and straightforward to implement, but the detection step requires more care.

To identify cases where the residue computation suffers from significant numerical error, we must determine whether a near-zero residue is the result of cancellation that hides a larger error. Simply checking whether the residue value is small is insufficient: we do not want to trigger residue override when a residue is zero because the computation is genuinely well behaved (for example, when a cancellation is mathematically expected and no absorption occurs). Formally, let the residue e_z be computed by the residue function e_f . RePo aims to trigger detection only when both

$$e_z \approx 0 \quad \text{and} \quad \text{NONTRIVIAL}(e_f)$$

hold. To evaluate these conditions, RePo employs two heuristics.

The first condition, $e_z \approx 0$, captures potential cancellations. An exactly zero residue is trivial to detect, but it is also useful to detect cases where the residue is very small but nonzero due to cancellation. To this end, RePo adds an `isZero` flag to each residue. This flag is set when either the residue is exactly zero or the condition number of the residue function,

$$\frac{|A\mu_z| + |Be_x| + |Ce_y|}{|e_z|},$$

exceeds a predefined large threshold, where $e_z = A\mu_z + Be_x + Ce_y$. Intuitively, if the contributing terms nearly cancel so that $|e_z| \ll |A\mu_z| + |Be_x| + |Ce_y|$, the ratio becomes large and the residue is flagged. This idea is similar to ATOMU's notion of atomic conditions [36], except that it applies to the debugger's shadow computations rather than to the program's original operations.

The second condition, `NONTRIVIAL`(e_f), distinguishes harmful cancellations from benign ones. Residue computations may cancel even when no rounding error has actually rounded away any error terms. Such cancellations are numerically correct, and residue override would be unnecessary. To avoid triggering on these benign cases, residue override introduces an `isAbsorbed` flag for each residue. The `isAbsorbed` flag is determined by comparing the largest error contribution (which determines `maxErrOp`) to the final residue value. If the largest contribution accounts for an overwhelming fraction—all but a few ULPs—of the final residue, `isAbsorbed` is set. When a cancellation occurs between two residues whose `isAbsorbed` flags are both clear, residue override treats the cancellation as benign and does not trigger override. Residue override triggers only when

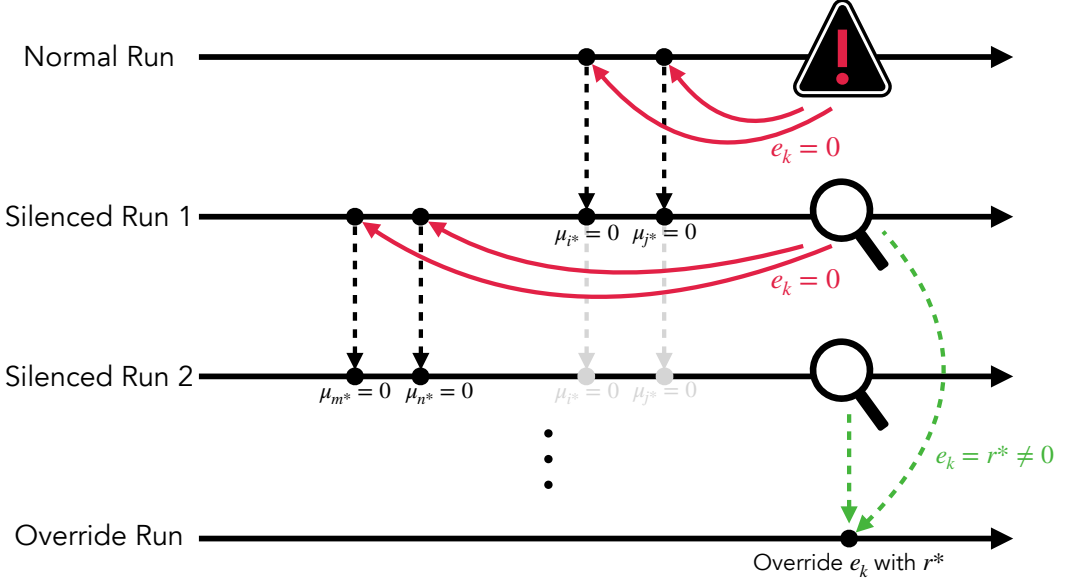


Fig. 3. Repeated silencing in RePo. Probing the first silenced run still flags e_k as an absorption and thus identifies two additional largest contributors, m^* and n^* . RePo then performs a second silenced run silencing all four identified largest contributors. Eventually, a nonzero value r^* is obtained for the residue e_k , allowing RePo to perform the final override run with the corrected value r^* .

a residue has both its `isZero` and `isAbsorbed` flags set, after which RePo identifies the leading contributors i^* and j^* to the two input residues and initiates the silence-probe-override procedure.

Both the cancellation condition and the nontriviality condition involve thresholds that must be set heuristically. Empirically, we find that adjusting these thresholds trades off between the number of re-executions that RePo performs and the number of residues that RePo can correct using residue override. We leave more principled approaches for identifying absorption-affected residues to future work; nevertheless, this simple heuristic approach already yields good results in Section 7.

6 Complex Absorption Scenarios

The implementation described in Section 5 resolves the simplest case of absorption: a single inaccurately computed residue that can be corrected by silencing one pair of operations. However, real-world numerical applications often exhibit more complex absorption patterns. To make residue override practical in such settings, RePo must address two additional challenges: absorptions that require silencing multiple pairs of operations, and programs that exhibit multiple independent absorptions. The solution builds on the same core residue override steps—detect, silence, probe, and override—but requires interleaving these stages.

6.1 Handling Multi-Term Absorption

Figure 3 illustrates an absorption scenario in which absorption is detected for residue e_k , the two largest contributors i^* and j^* are silenced, but, when probed, e_k is still detected to suffer from absorption. This occurs because absorption can affect arbitrarily large sets of mutually immeasurable

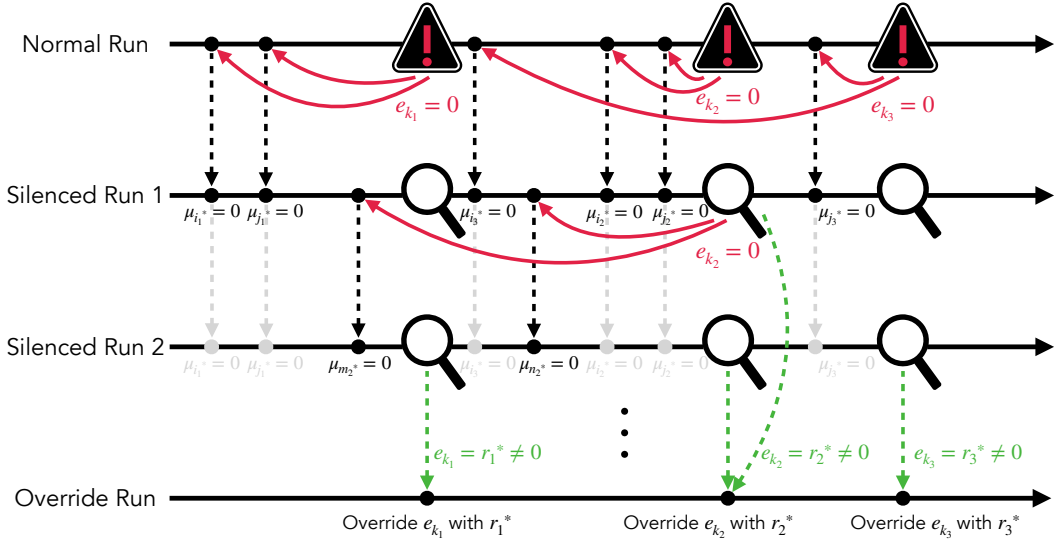


Fig. 4. Handling multiple absorptions simultaneously. During the initial run, three residues (k_1 , k_2 , and k_3) exhibit absorption. In subsequent silenced runs, residue override successively silences the dominant contributors (i^* , j^* , etc.) and recomputes the affected residues until each residue obtains a nonzero value (r_1^* , r_2^* , r_3^*). Finally, in the override run, residue override replaces the corresponding residues e_{k_1} , e_{k_2} , and e_{k_3} with the corrected values r_1^* , r_2^* , and r_3^* .

residues. Such patterns are particularly common in complex routines, such as implementations of the sine function.

To handle these cases, RePo continues to track the largest contributor to each residue *even during the silenced run*, and correspondingly updates the `maxErrOp` field. In the silenced run, some rounding errors are set to zero, and are therefore no longer the largest contributors to any residue, even if they were during the initial run. Thus, if absorption is detected at e_k during the silenced run, two *additional* operations m^* and n^* are identified as contributing to the absorption. RePo then performs *another* silenced run in which i^* , j^* , m^* , and n^* are all silenced, and, if absorption is still detected, further runs silencing additional operations can be performed. This process stops when e_k is no longer detected as suffering from absorption, meaning the `isZero` or `isAbsorbed` flag of e_k is no longer set.

6.2 Handling Multiple Independent Absorptions

Multi-term absorptions still affect only a *single* residue in a program. However, in real-world numerical software, there are often *multiple* residues affected by absorption. These residues may be independent of one another (discussed in this subsection) or, worse, may interact (discussed in the next one). When the residues are independent, RePo attempts to resolve all absorptions within the same re-execution.

At a high level, RePo does so by performing multiple silences and probes simultaneously, as shown in Figure 4. In the first execution of RePo, three residues are affected by absorption: k_1 , k_2 , and k_3 . For each one, RePo identifies its largest contributors: i_1^* , j_1^* for k_1 , i_2^* , j_2^* for k_2 , and i_3^* , j_3^* for k_3 . In the next run, all six largest contributors are silenced, and more accurate values of k_1 , k_2 , and k_3 are probed. It is possible that some of the probed residues are still affected by absorption, such as k_2 in the figure; in this case, RePo silences the next pair of largest contributors, as in the case of

multi-term absorptions. Finally, once all probed values are accurate, RePo proceeds to the override stage.

Practically, to handle multiple absorptions, RePo maintains four data structures across runs:

- `silentOps`, the set of operations whose rounding errors should be silenced;
- `probeOps`, the set of operations whose residues should be probed;
- `tempResOverride`, a temporary mapping from probed operations i to their newly computed residues r_i^* ;
- `resOverride`, a permanent mapping from operations to their accurately computed residues.

All four data structures are stored on disk, loaded by RePo at the beginning of each re-execution, and used and updated according to this logic:

```
void compute_residue(shadow_value_t *e_z) {
  if (resOverride.contains(curOp)) { *e_z = resOverride[curOp]; }
  if (silentOps.contains(curOp)) { introErr = 0; }
  else { // Normal error-free transformation routine }
  // Compute e_z
  if (probeOps.contains(curOp)) { tempResOverride[curOp] = e_z->value; }
}
```

Between re-executions, operations involved in absorptions are added to `silentOps` and `probeOps`; entries in `probeOps` are moved to `tempResOverride`; and entries in `tempResOverride` are moved to `resOverride` if no additional absorptions are detected. This scheme can efficiently handle multiple absorptions in as few as three executions.

6.3 Second-Largest Contributor for Parallel Resolution

Finally, when multiple residues suffer from absorption, it is also possible for those residues to interact with one another. The first, simpler case of interaction is when accurately computing one residue causes later residues to suffer from absorption. This is common, because residues that suffer from absorption are typically very small, and computing those residues more accurately often makes them larger, meaning that later residues may acquire new, dominant contributors. RePo detects this case when, during a silence or override run, a residue that did not *previously* suffer from absorption begins to suffer from it. In this case, RePo adds the new residue's largest contributors to the silenced set, the new residue to the probed set, and performs another re-execution.

The more challenging case of interaction is when two different residues both suffer from absorption, but silencing one residue's largest contributor affects the probed value of the other residue. For example, an important, but not largest, contributor to one absorption may also be the largest contributor to another absorption. Silencing this contributor (to probe the second absorption) would remove an important contribution for the first absorption, causing the probe of the first absorption to measure an incorrect value.

In this case, the two residues cannot be accurately measured in the same re-execution. RePo therefore detects this issue and measures them in separate re-executions. To detect this case, RePo tracks the *second-largest* contributor to each residue, denoted `sndErrOp`. It is computed together with the largest contributor `maxErrOp`, as follows:

```
if (absIntroErr >= max(absAmpErr1, absAmpErr2)) {
  e_z->sndErrOp = (absAmpErr1 > absAmpErr2) ? e_x->maxErrOp : e_y->maxErrOp;
}
else if (absAmpErr1 >= absAmpErr2) {
  e_z->sndErrOp = (absIntroErr > absAmpErr2) ? curOp : e_y->maxErrOp;
```

```

}
else {
    e_z->sndErrOp = (absIntroErr > absAmpErr1) ? curOp : e_x->maxErrOp;
}

```

The key distinction between the largest and second-largest contributors is that the largest contributor determines *which* operation to silence, while the second-largest contributor determines *whether* two absorptions can be resolved simultaneously. When multiple absorptions are detected, RePo iterates through them, adding the largest contributors of each absorption to the silenced set *only* if their second-largest contributors are not already present in the silenced set. After a successful silencing-and-probing run, some absorptions are resolved, their largest contributors are removed from the silenced set, and later interacting residues are then silenced and probed.

6.4 Summary

The final data structure of a residue in RePo is as follows:

```

struct Residue {
    double value; // computed residue
    OpId maxErrOp; // largest contributor operation ID
    OpId sndErrOp; // second-largest contributor operation ID
    bool isAbsorbed; // absorption flag
    bool isZero; // zero flag
};

```

In other words, each residue stores its computed value, its largest and second-largest contributor operations, and its two flags. These residues are then organized using the `silentOps`, `probeOps`, and `resOverride` sets, which are updated as described in Algorithm 1. This algorithm enables RePo to resolve complex, real-world absorption scenarios with as few re-executions as possible.

7 Evaluation

Even without residue override, RePo is substantially more accurate than existing state-of-the-art debuggers on standard scientific benchmark suites, and residue override rarely triggers on them. To evaluate the impact of residue override, we therefore focus on more challenging numerical benchmarks. We aim to answer three research questions:

- RQ1: Does residue override improve RePo's false report rate?
- RQ2: How many re-executions does residue override require?
- RQ3: How does residue override compare to high-precision arithmetic?

We draw challenging numerical programs from two suites: EFTSanitizer's 45-program test suite [8], which is drawn from prior floating-point debugging work, and the standard FPBench [10] suite, which contains 130 numerical kernels collected from numerical analysis papers and textbooks. The FPBench benchmarks are expressions rather than complete programs, so we compile them into C functions using the FPBench toolchain and generate simple driver programs to invoke these functions with random inputs (using a fixed seed for determinism).⁶ Six benchmarks run out of memory when computing ground-truth MPFR residues, leaving a total of 169 benchmarks. We focus on a subset of 34 benchmarks for which the first run of RePo produces false reports. These benchmarks represent the most challenging cases and are also those where residue override is

⁶We exclude the apron suite, which the FPBench toolchain cannot compile to C. Each driver generates 500 inputs spanning a range of exponents and executes the kernel on all 500 inputs in a loop.

Algorithm 1 Parallel Residue Override

```

procedure RESOLVE(p, silentOps, probeOps, resOverride)
  repeat
    stillCancel  $\leftarrow$  false
    maxErrOps, sndErrOps  $\leftarrow$   $\emptyset$ 
    tempResOverride, absorptions  $\leftarrow$  EXECUTE(p, silentOps, probeOps, resOverride)
    for  $(i_x^*, j_x^*, i_y^*, j_y^*, k) \in$  absorptions do
      if  $(k \in \text{probeOps}) \wedge (i_x^*, i_y^* \cap \text{sndErrOps} = \emptyset) \wedge (j_x^*, j_y^* \cap \text{maxErrOps} = \emptyset)$  then
        stillCancel  $\leftarrow$  true
        silentOps.ADD( $i_x^*, i_y^*$ )
        maxErrOps.ADD( $i_x^*, i_y^*$ )
        sndErrOps.ADD( $j_x^*, j_y^*$ )
      end if
    end for
  until not stillCancel
  return tempResOverride
end procedure

procedure REPO(p) ▷ driver program
  hasCancel  $\leftarrow$  false
  silentOps, probeOps, resOverride  $\leftarrow$   $\emptyset$ 
  maxErrOps, sndErrOps  $\leftarrow$   $\emptyset$ 
  while true do
    _, absorptions  $\leftarrow$  EXECUTE(p, silentOps, probeOps, resOverride)
    hasCancel  $\leftarrow$  false
    for  $(i_x^*, j_x^*, i_y^*, j_y^*, k) \in$  absorptions do
      if  $(\{i_x^*, i_y^*\} \cap \text{sndErrOps} = \emptyset) \wedge (\{j_x^*, j_y^*\} \cap \text{maxErrOps} = \emptyset)$  then
        hasCancel  $\leftarrow$  true
        silentOps.ADD( $i_x^*, i_y^*$ )
        probeOps.ADD( $k$ )
        maxErrOps.ADD( $i_x^*, i_y^*$ )
        sndErrOps.ADD( $j_x^*, j_y^*$ )
      end if
    end for
    if not hasCancel then
      break
    end if
    tempResOverride  $\leftarrow$  RESOLVE(p, silentOps, probeOps, resOverride)
    resOverride.UPDATE(tempResOverride)
    CLEAR(silentOps, probeOps, maxErrOps, sndErrOps)
  end while
  return
end procedure

```

expected to have an effect.⁷ Several benchmarks involve math library functions such as sin, exp, or log, whose implementations are particularly challenging to analyze. We cap RePo's residue override algorithm at 20 re-executions and perform all experiments on a machine with Ubuntu 24.04.3, an Intel i7-8700K at 3.70 GHz, and 32 GB of memory, using Clang 18.1.3 for all compilation.

RQ1. Among the 34 benchmarks, residue override triggers on 29 of them. Figure 5 shows their initial and final false reports. On 25 benchmarks (86.2%), residue override reduces the total number of false reports, often dramatically.

⁷The bug-fixed re-implementation of EFTSanitizer produces false reports on all 34 benchmarks as well as 18 additional benchmarks.

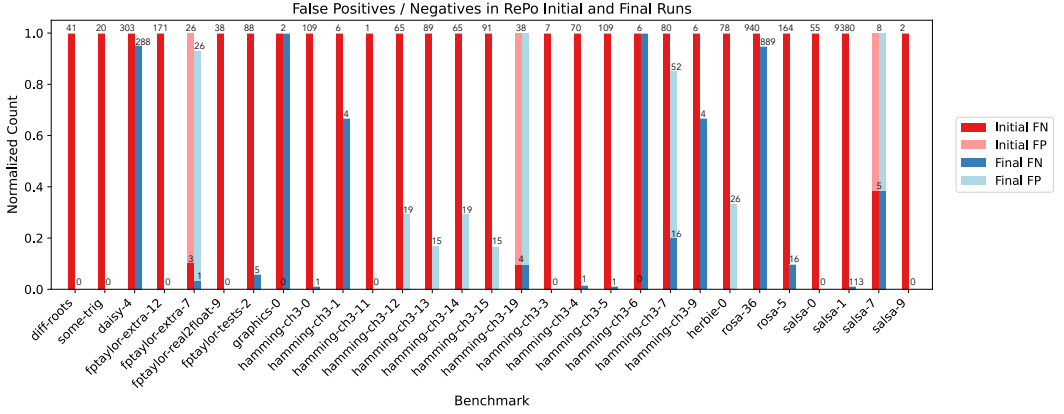


Fig. 5. Number of false reports (false positives and false negatives) for the initial and final runs of RePo. Only benchmarks for which RePo has non-perfect initial results are shown; all bars are normalized so that the total height of the initial run is 1.0. RePo produces fewer false reports on 25 out of 29 benchmarks. On 19 of these, the residue override mechanism reduces the number of false negatives by about 10x compared to the initial run.

Among these 25 benchmarks, residue override improves both false positives and false negatives on 19 of them, reducing at least one without increasing the other. Specifically, 8 of them achieve perfect results (no false reports) in their final runs. For example, the `diff-roots` benchmark, which is used as the running example in Section 2, evaluates $\sqrt{x+1} - \sqrt{x}$ on 100 positive inputs with varying exponents. RePo’s initial run produces 41 false negatives, but residue override detects and corrects all inaccurate residues, achieving perfect results in the final override run. Moreover, some benchmarks would achieve perfect results with a higher cap on re-executions. For example, `hamming-ch3-1` in the FPBench suite, which computes $\sin(x + \epsilon) - \sin(x)$, resolves two of six false negatives within the first 20 re-executions, but resolves the remaining four false negatives with just three additional re-executions. This benchmark requires many re-executions because it must handle complex absorptions arising from multi-precision arithmetic inside the sine function implementation.

For 6 out of 25 benchmarks, residue override reduces the number of false negatives but introduces some false positives. For example, on the `hamming-ch3-14` benchmark, residue override eliminates all 65 false negatives from the first run but introduces 19 false positives. Even so, in these cases, residue override reduces both the total number of false reports and, in particular, the number of false negatives, which are the most dangerous type of error for a debugger.

On 4 benchmarks, residue override does not change the number of false reports. However, for two of them, `graphics-0` and `hamming-ch3-6`, this is due to the re-execution cap; increasing the cap allows both benchmarks to achieve perfect results but requires 50 re-executions. We expect that future work can examine more efficient methods for reasoning about elementary function implementations that reduce the number of required re-executions.

In summary, **RePo’s residue override mechanism significantly reduces false reports** even on the most challenging numerical benchmarks.

RQ2. While residue override is effective, it requires multiple executions of the program, introducing additional debugging overhead. We therefore measure how many re-executions are required for RePo’s residue override algorithm to terminate. Figure 6 shows the results as a histogram.

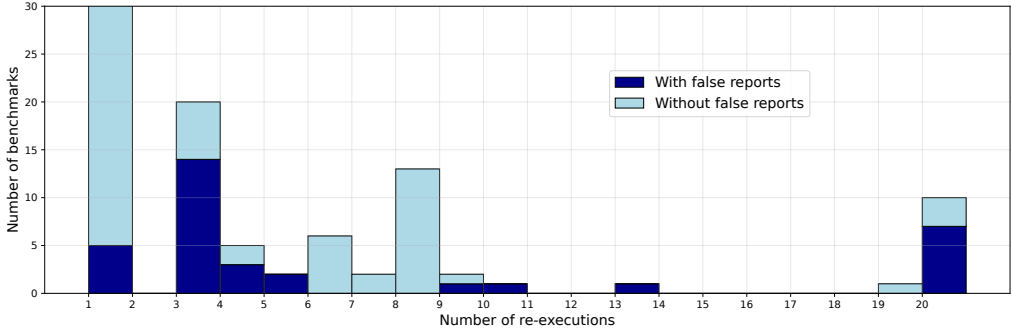


Fig. 6. Number of re-executions for all 169 benchmarks (34 benchmarks with false reports in their initial runs, and 135 benchmarks without false reports in their initial runs). We cap the plot at 30 benchmarks to emphasize benchmarks whose initial runs have false reports. For benchmarks without false reports (already accurate), the distribution of re-executions is highly right-skewed: 101 out of 135 benchmarks require no re-executions. For benchmarks with false reports, residue override is triggered on most of them, and nearly half require only three executions, thanks to RePo’s ability to handle multiple absorptions simultaneously.

Across all 169 benchmarks, RePo requires 3.6 re-executions on average, with 106 benchmarks requiring no re-executions because no absorption is detected. For the remaining 63 benchmarks, RePo requires at least 3 re-executions: one for the initial detection run, one for the silenced run, and one for the final override run. Twenty benchmarks require only these 3 re-executions, while the remaining benchmarks require more, often due to calls to library functions such as `sin`, which introduce deeper chains of cancellation and therefore require additional re-executions. Ten benchmarks reach the cap of 20 re-executions.

Benchmarks *without* false reports in their initial runs require very few re-executions in Figure 6: on these benchmarks, RePo requires 2.7 re-executions on average; in contrast, on the 34 benchmarks considered in RQ1, where the initial runs produce false reports, it requires 7.1 re-executions. This highlights an advantage of residue override over higher-precision floating-point debuggers: for already accurate applications, RePo incurs very low overhead. We explore RePo’s overhead further in RQ3.

Some benchmarks trigger the residue override mechanism but produce the same final results as the initial run. This occurs when the detected absorptions do not affect whether the computed residues cross the reporting threshold. While these cases do not improve the reported results, more sophisticated floating-point debuggers may still benefit from the improved residue accuracy.

In summary, **RePo requires an acceptable number of re-executions**, even for challenging numerical benchmarks.

RQ3. The results above demonstrate that RePo significantly improves on EFTSanitizer, with further improvements on challenging numerical benchmarks enabled by residue override. However, although EFTSanitizer is the current state of the art among floating-point debuggers that use machine-float residues, other debuggers instead use higher-precision residues. We therefore compare RePo, with residue override enabled, to both the ground-truth MPFR-based debugger and a debugger based on the QD library, which computes residues from quad-double shadow values.

Among the three debuggers, MPFR is the most accurate by definition, since we use it to obtain the ground-truth results. The comparison between RePo and QD is therefore more interesting. Across all evaluated benchmarks, RePo and QD differ on 57 benchmarks; RePo achieves better results on 43 of them (75.44%), 6 exhibit a mixed pattern of improvements and regressions (one debugger

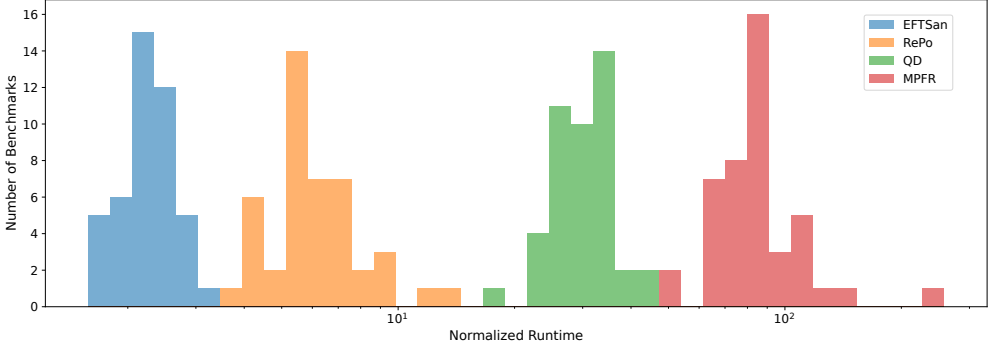


Fig. 7. Distribution of runtime overhead for EFTSanitizer, RePo, QD, and MPFR, normalized to uninstrumented execution. The x-axis is logarithmic to capture the wide range of overheads. Overall, RePo exhibits overhead somewhat higher than EFTSanitizer but strictly lower than QD, and substantially lower than MPFR. RePo is more accurate than QD and substantially more accurate than EFTSanitizer, placing it at a particularly practical point in the speed-accuracy spectrum.

produces more false positives while the other produces more false negatives), while QD achieves better results on just 8. In other words, lower-precision residues combined with residue override provide better accuracy, on average, than higher-precision residues without residue override.

Moreover, RePo is also faster than the QD or MPFR debuggers. We instrumented each debugger to measure only the execution time of floating-point operations (including all shadow operations or residue computations) using an `rdtsc`-based cycle counter. Figure 7 shows the runtime overhead, compared to uninstrumented execution, across all scientific benchmark suites for the EFTSanitizer, RePo, QD, and MPFR debuggers. Note that the horizontal axis is logarithmic. All debuggers are slower than uninstrumented execution: EFTSanitizer is the fastest, RePo is somewhat slower, while QD and MPFR are slower still. This overhead arises from the cost of performing each floating-point operation using software-based high-precision arithmetic. Although RePo is slower than EFTSanitizer, due to the additional operation tracking required for residue override, it is substantially more accurate, placing RePo at a favorable balance between high accuracy and low overhead.

Sensitivity to Thresholds. The main heuristic parameter in residue override is the absorption threshold τ_{abs} , which determines when the largest error contribution accounts for enough of the final residue for a cancellation to be treated as absorption. To evaluate sensitivity to this parameter, we tested all 169 benchmarks using the default threshold from our main experiments, together with two nearby settings.⁸ Overall, the behavior is quite stable. All 45 microbenchmarks are unchanged across the sweep, and in FPBench only 7 benchmarks change in either their final false reports or their number of re-executions. As the threshold decreases, residue override becomes more aggressive: the average number of re-executions per benchmark rises modestly from 3.57 at the highest setting to 3.75 at the lowest. At the same time, the final false-report results improve monotonically: false negatives decrease, while false positives do not increase. These results suggest that residue override is robust to nearby threshold choices, and that the default setting provides a reasonable balance between recovering missed absorptions and avoiding additional re-executions.

⁸Relative to the default threshold, we evaluated one setting 2 machine epsilons larger and one 7 machine epsilons smaller.

8 Related Work

Numerical analysis and the correctness of numerical software have long been studied in computer science [19, 21]. More recently, researchers have developed automated tools for deriving error bounds or finding bugs in numerical programs.

Error Bound Tools. Some of the earliest analysis tools for numerical software used abstract interpretation to bound rounding error. For example, Salsa [9] represented program states using pairs of intervals, one for variable values and one for errors. Conceptually, this resembles a residue-based technique, using interval arithmetic to bound residues. Later tools followed the same separation between values and errors but used different methods to bound the error intervals. For example, Rosa [11] and Daisy [20] used affine arithmetic to obtain tighter error bounds. The FPTaylor project [32], likewise, modeled values and errors separately using an “error Taylor series”—a generalization of affine arithmetic—and then used a nonlinear global optimizer to bound the maximum error. Later tools such as Satire [12] follow a similar approach. RePo uses a similar residue-based idea, though it performs its analysis dynamically at runtime. In contrast, error-bound tools perform their analysis symbolically, compute ranges rather than specific error values, and optimize for tight error bounds rather than runtime overhead. More broadly, symbolic error-analysis tools and floating-point debuggers serve complementary roles: RePo can help localize numerically unstable executions and kernels in large applications, after which symbolic tools can be used to derive stronger guarantees for those kernels.

More recently, the NumFuzz approach [25] has explored faster numerical analysis using techniques inspired by type checking, including extensions to backward error analysis [22, 35]. This type-theoretic approach differs substantially from the value/error separation used by previous tools, focusing on error sensitivity rather than the exact range of values or errors. However, this technique has not yet been extended to handle subtraction and cancellation, let alone complex scientific software.

One challenge for many numerical analysis techniques is control-flow divergence between the floating-point execution and the ideal real execution. For example, Seesaw [13] analyzes conditional expressions to determine whether numerical error could cause a conditional to evaluate differently. This problem is especially difficult for analysis tools, which must consider a range of inputs. Floating-point debuggers like RePo, by contrast, typically follow the floating-point control flow while reporting potential points of divergence.

Floating-Point Debugging. In contrast to analysis tools, floating-point debuggers execute the program on specific inputs and identify cases where rounding error accumulates enough to significantly deviate from the ideal real-number result. Because they analyze a specific execution, floating-point debuggers can often be more precise: they can explicitly detect control-flow divergence and compute exact error values for intermediate variables.

Early debuggers such as FpDebug [3] used arbitrary-precision shadow values to detect operations like subtraction that cause cancellation. This idea was developed further in Herbgrind [31], which introduced the concept of *spots*—program outputs or control-flow decisions where floating-point error produces user-visible effects. Herbgrind then applies anti-unification to identify a minimal floating-point expression tree responsible for these effects. However, these early debuggers were relatively slow because they relied on arbitrary-precision arithmetic.

Later work focused on improving performance. FPSanitizer [6] integrated shadow computations and error reporting directly into the compilation pipeline as an LLVM pass and also explored multi-threaded debugging to reduce overhead [7]. However, arbitrary-precision arithmetic remained a bottleneck. EFTSanitizer addressed this by replacing arbitrary-precision shadow values with

residues computed directly using machine floating-point operations and error-free transformations. Combined with FPSanitizer's optimized compilation process, EFTSanitizer achieved overheads of about 10× while also computing dependency graphs to help developers understand error propagation. Similarly, Shaman [14] uses error-free transformations to identify values with high numerical error, though it requires users to instrument specific parts of the program. However, abandoning arbitrary precision leads to higher rates of false positives and false negatives, a gap that RePo aims to close.

Meanwhile, several tools avoid shadow-value computation entirely. For example, ATOMU [36] computes condition numbers for each floating-point operation and issues warnings when the condition number is large. Because ATOMU does not track shadow values, it avoids the question of how precise those values must be. Explanifloat [23] extends this idea further, combining error-free transformations, a logarithmic number system, and condition numbers to detect subtle rounding errors. However, condition-number-based approaches tend to produce many false reports. Explanifloat, for example, reports a precision of about 80%, meaning that roughly 20% of its warnings are false.

The BZ [1] and RAIVE [24] tools also avoid storing shadow values, instead inspecting the exponents of computed floating-point values to detect potential numerical issues. These approaches likewise suffer from high false-report rates. Finally, the Verrou project [15] uses stochastic rounding, effectively perturbing floating-point values, to study the impact of rounding error. Like RePo, this approach relies on repeated executions to understand numerical error. However, Verrou aggregates these executions statistically, while RePo identifies specific cancellations and re-executes to recover the hidden residue terms responsible for them. This makes RePo's re-executions more targeted and gives its corrected residues a more direct connection to the real-number computation. Mixed-precision tuning tools such as Precimonious [30] and FPTuner [5] likewise reason about numerical error, but their goal is to choose precisions rather than explain a particular inaccurate execution.

9 Conclusion

Floating-point debuggers can help programmers identify and, ultimately, fix numerical issues. However, even the best existing debuggers have limited accuracy, which can make their results misleading. In this work, we improved residue-based debugging at two levels. First, we refined the machinery for computing machine-float residues: how rounding errors are estimated and how those errors are propagated through residue functions. These changes made the debugger's shadow computation more faithful to the underlying real-number execution, reducing false reports in all of the scientific workloads where prior tools produce misleading results, without sacrificing the efficiency of machine-precision shadow values.

We then showed that some remaining failures were not merely due to bugs or approximations in residue formulas, but stemmed from a more fundamental limitation of fixed-precision residues. Under absorption, a residue could preserve the dominant contribution needed at one point in the computation or the smaller contribution needed after a later cancellation, but not both within a single execution. The residue override framework addressed this limitation by separating these measurements across multiple runs: silenced runs revealed information that ordinary execution hides, and the final override run reintroduced corrected residues at the points where they are needed. The idea was further extended to handle more complex cases that arise in practice, including repeated silencing and multiple interacting cancellations. On 34 benchmarks drawn from numerical analysis papers and textbooks where initial runs failed to achieve perfect results, residue override reduced false report counts in 25 of them while requiring only a modest number of re-executions on average. Overall, these results show that substantially more accurate floating-point debugging does

not require high-precision shadow computation. Instead, improved residue computation, combined with targeted override runs for absorption, recovers much of that accuracy at far lower cost.

Data Availability Statement

The code and data used in this paper are available on Zenodo [17], and the source code repository is available on GitHub [18]. The benchmarks used in our evaluation are all publicly available, though we do not own them; they come from the NAS Parallel Benchmarks [2], Rodinia [4], PolyBench [28], EFTSanitizer microbenchmark suite [8], and FPBench [10].

Acknowledgments

This work was supported by the National Science Foundation under Grant No. CNS-2346394. We thank the anonymous reviewers for their feedback and Santosh Nagarakatte for valuable comments about the work.

References

- [1] Tao Bao and Xiangyu Zhang. 2013. On-the-fly Detection of Instability Problems in Floating-point Program Execution. *SIGPLAN Not.* 48, 10 (Oct. 2013), 817–832. doi:10.1145/2544173.2509526
- [2] NAS Parallel Benchmarks. 2006. Nas parallel benchmarks. *CG and IS* (2006).
- [3] Florian Benz, Andreas Hildebrandt, and Sebastian Hack. 2012. A dynamic program analysis to find floating-point accuracy problems. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation* (Beijing, China) (*PLDI '12*). Association for Computing Machinery, New York, NY, USA, 453–462. doi:10.1145/2254064.2254118
- [4] Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W. Sheaffer, Sang-Ha Lee, and Kevin Skadron. 2009. Rodinia: A benchmark suite for heterogeneous computing. In *2009 IEEE International Symposium on Workload Characterization (IISWC)*. 44–54. doi:10.1109/IISWC.2009.5306797
- [5] Wei-Fan Chiang, Mark Baranowski, Ian Briggs, Alexey Solovvey, Ganesh Gopalakrishnan, and Zvonimir Rakamarić. 2017. Rigorous Floating-point Mixed-precision Tuning (*POPL*). 300–315. doi:10.1145/3009837.3009846
- [6] Sangeeta Chowdhary, Jay P. Lim, and Santosh Nagarakatte. 2020. Debugging and detecting numerical errors in computation with posits. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation* (London, UK) (*PLDI 2020*). Association for Computing Machinery, New York, NY, USA, 731–746. doi:10.1145/3385412.3386004
- [7] Sangeeta Chowdhary and Santosh Nagarakatte. 2021. Parallel shadow execution to accelerate the debugging of numerical errors. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Athens, Greece) (*ESEC/FSE 2021*). Association for Computing Machinery, New York, NY, USA, 615–626. doi:10.1145/3468264.3468585
- [8] Sangeeta Chowdhary and Santosh Nagarakatte. 2022. Fast shadow execution for debugging numerical errors using error free transformations. *Proceedings of the ACM on Programming Languages* 6, OOPSLA2 (2022), 1845–1872. doi:10.1145/3563353
- [9] Nasrine Damouche and Matthieu Martel. 2018. Salsa: An Automatic Tool to Improve the Numerical Accuracy of Programs. In *Automated Formal Methods (Kalpa Publications in Computing, Vol. 5)*, Natarajan Shankar and Bruno Dutertre (Eds.). EasyChair, 63–76. doi:10.29007/j2fd
- [10] Nasrine Damouche, Matthieu Martel, Pavel Panchekha, Chen Qiu, Alexander Sanchez-Stern, and Zachary Tatlock. 2016. Toward a standard benchmark format and suite for floating-point analysis. (2016), 63–77. doi:10.1007/978-3-319-54292-8_6
- [11] Eva Darulova and Viktor Kuncak. 2014. Sound compilation of reals. In *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 235–248. doi:10.1145/2535838.2535874
- [12] Arnab Das, Ian Briggs, Ganesh Gopalakrishnan, Sriram Krishnamoorthy, and Pavel Panchekha. 2020. Scalable yet Rigorous Floating-Point Error Analysis. In *2020 SC20: International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. IEEE Computer Society, Los Alamitos, CA, USA, 1–14. doi:10.1109/SC41405.2020.00055
- [13] Arnab Das, Tanmay Tirpankar, Ganesh Gopalakrishnan, and Sriram Krishnamoorthy. 2021. Robustness Analysis of Loop-Free Floating-Point Programs via Symbolic Automatic Differentiation. In *2021 IEEE International Conference on Cluster Computing (CLUSTER)*. 481–491. doi:10.1109/Cluster48925.2021.00055

- [14] Nestor Demeure, Cédric Chevalier, Christophe Denis, and Pierre Dossantos-Uzarralde. 2023. Algorithm 1029: Encapsulated Error, a Direct Approach to Evaluate Floating-Point Accuracy. *ACM Trans. Math. Software* 48, 4 (2023), 1–16. doi:10.1145/3549205
- [15] François Févotte and Bruno Lathuilière. 2016. VERROU: Assessing Floating-Point Accuracy Without Recompiling. (Oct. 2016). <https://hal.archives-ouvertes.fr/hal-01383417>
- [16] Laurent Fousse, Guillaume Hanrot, Vincent Lefèvre, Patrick Pélissier, and Paul Zimmermann. 2007. MPFR: A Multiple-Precision Binary Floating-Point Library with Correct Rounding. *ACM Trans. Math. Software* 33, 2 (June 2007), 13:1–13:15. doi:10.1145/1236463.1236468
- [17] Yumeng He and Pavel Panchekha. 2026. *Artifact for Accurate Residues for Floating-Point Debugging*. doi:10.5281/zenodo.21437395
- [18] Yumeng He and Pavel Panchekha. 2026. RePo. GitHub repository. <https://github.com/ymh1003/RePo>.
- [19] Nicholas J. Higham. 2002. *Accuracy and Stability of Numerical Algorithms* (second ed.). Society for Industrial and Applied Mathematics. arXiv:<https://epubs.siam.org/doi/pdf/10.1137/1.9780898718027> doi:10.1137/1.9780898718027
- [20] Anastasiia Izycheva and Eva Darulova. 2017. On sound relative error bounds for floating-point arithmetic (FMCAD). 15–22. doi:10.23919/FMCAD.2017.8102236
- [21] William Kahan. 1983. Mathematics written in sand. In *Proc. Joint Statistical Mtg. of the American Statistical Association*. Citeseer, 12–26.
- [22] Ariel E. Kellison, Laura Zielinski, David Bindel, and Justin Hsu. 2025. Bean: A Language for Backward Error Analysis. *Proc. ACM Program. Lang.* 9, PLDI, Article 221 (June 2025), 25 pages. doi:10.1145/3729324
- [23] Bhargav Kulkarni and Pavel Panchekha. 2025. Mixing Condition Numbers and Oracles for Accurate Floating-point Debugging. In *2025 IEEE 32nd Symposium on Computer Arithmetic (ARITH)*. 101–108. doi:10.1109/ARITH64983.2025.00025
- [24] Wen-Chuan Lee, Tao Bao, Yunhui Zheng, Xiangyu Zhang, Keval Vora, and Rajiv Gupta. 2015. RAIVE: runtime assessment of floating-point instability by vectorization. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (Pittsburgh, PA, USA) (OOPSLA 2015)*. Association for Computing Machinery, New York, NY, USA, 623–638. doi:10.1145/2814270.2814299
- [25] Chenghu Ma, Liqian Chen, Xin Yi, Guangsheng Fan, and Ji Wang. 2022. NuMFUZZ: A Floating-Point Format Aware Fuzzer for Numerical Programs. In *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*. 338–347. doi:10.1109/APSEC57359.2022.00046
- [26] B. D. McCullough and H. D. Vinod. 1999. The Numerical Reliability of Econometric Software. *Journal of Economic Literature* 37, 2 (1999), 633–665. doi:10.1257/jel.37.2.633
- [27] Jean-Michel Muller, Nicolas Brisebarre, Florent de Dinechin, Claude-Pierre Jeannerod, Vincent Lefèvre, Guillaume Melquiond, Nathalie Revol, Damien Stehlé, and Serge Torres. 2009. *Handbook of Floating-Point Arithmetic* (1st ed.). Birkhäuser Basel.
- [28] Louis-Noël Pouchet. 2012. Polybench/C. <https://www.cs.colostate.edu/~pouchet/software/polybench/>
- [29] Kevin Quinn. 1983. Ever Had Problems Rounding Off Figures? This Stock Exchange Has. *The Wall Street Journal* (Nov. 1983), 37.
- [30] Cindy Rubio-González, Cuong Nguyen, Hong Diep Nguyen, James Demmel, William Kahan, Koushik Sen, David H Bailey, Costin Iancu, and David Hough. 2013. Precimonious: Tuning assistant for floating-point precision. In *Proceedings of the international conference on high performance computing, networking, storage and analysis*. 1–12. doi:10.1145/2503210.2503296
- [31] Alex Sanchez-Stern, Pavel Panchekha, Sorin Lerner, and Zachary Tatlock. 2018. Finding Root Causes of Floating Point Error (PLDI). 256–269. doi:10.1145/3192366.3192411
- [32] Alexey Solovyev, Marek S Baranowski, Ian Briggs, Charles Jacobsen, Zvonimir Rakamarić, and Ganesh Gopalakrishnan. 2018. Rigorous estimation of floating-point round-off errors with symbolic taylor expansions. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 41, 1, 1–39. doi:10.1145/3230733
- [33] U.S. General Accounting Office. 1992. Patriot Missile Defense: Software Problem Led to System Failure at Dhahran, Saudi Arabia. <http://www.gao.gov/products/IMTEC-92-26>
- [34] Debora Weber-Wulff. 1992. Rounding error changes Parliament makeup. <http://catless.ncl.ac.uk/Risks/13.37.html#subj4>
- [35] Laura Zielinski and Justin Hsu. 2026. Synthesizing Backward Error Bounds, Backward. *Proceedings of the ACM on Programming Languages* 10, PLDI (2026), 2157–2180. doi:10.1145/3808333
- [36] Daming Zou, Muhan Zeng, Yingfei Xiong, Zhoulai Fu, Lu Zhang, and Zhendong Su. 2019. Detecting floating-point errors via atomic conditions. *Proc. ACM Program. Lang.* 4, POPL, Article 60 (dec 2019), 27 pages. doi:10.1145/3371128

Received 2026-03-17; accepted 2026-06-10